Morse Code Translation Project: Development Approach

Prepared for Project Development

August 9, 2025

Contents

1 Introduction

This report outlines a step-by-step approach to developing a Morse code translation system that converts English text to Morse code and vice versa, recommends similar corrected words for invalid Morse inputs using similarity metrics (e.g., Levenshtein distance), and processes input from a live camera or video using computer vision (e.g., OpenCV). The approach covers learning key topics with official resources, gathering datasets and sources, and progressing through design, implementation, and deployment.

2 Step 1: Learning Key Topics

To build the necessary skills, allocate 1–2 weeks for self-paced learning through official documentation and tutorials. Practice with small code snippets to reinforce concepts.

- **Programming Fundamentals (Python)**: Learn variables, data types, control structures (loops, if-else), functions, and string manipulation (splitting, joining, case conversion).
  - Official Python Documentation: https://docs.python.org/ (Tutorial section).
  - Python.org Guides: https://www.python.org/doc/ for downloads and resources.
- **Morse Code Basics**: Understand the International Morse Code standard, including dots (.), dashes (-), spaces for letters, and slashes (/) for words. Learn mappings for A-Z, 0-9, and punctuation.
  - ITU Recommendation: https://www.itu.int/rec/R-REC-M.1677-1-200910-I/ (PDF with standard chart).
  - Morse Code World: https://morsecode.world/international/morse2.html (Interactive tables).
- **Input/Output Handling & Error Handling**: Learn reading user input, validating strings (e.g., regex for Morse: ., -, /, spaces), and handling exceptions.
  - Python Regex Docs: https://docs.python.org/3/library/re.html.
  - Python Errors Tutorial: https://docs.python.org/3/tutorial/errors.html.
- **String Similarity & Recommendations (Levenshtein Distance)**: Study algorithms for string similarity to recommend words for invalid Morse inputs.
  - Python-Levenshtein Docs: https://pypi.org/project/python-Levenshtein/ (Install: `pip install python-Levenshtein`).
  - RapidFuzz Levenshtein: https://rapidfuzz.github.io/Levenshtein/.
  - GeeksforGeeks Intro: https://www.geeksforgeeks.org/python/introduction-to-pytho
- **Computer Vision for Visual Input**: Learn image processing, video capture, and signal detection (e.g., brightness changes) using OpenCV.

  - Official OpenCV Docs: https://docs.opencv.org/4.x/ (Tutorials section).
  - OpenCV.org: https://opencv.org/ for downloads and courses.
- **User Interface Development (Optional GUI)**: Build interfaces for input/output using Tkinter for desktop apps.
  - Tkinter Docs: https://docs.python.org/3/library/tkinter.html.
  - TkDocs: https://www.tkdocs.com/ for Tk tutorials.
- **Deployment Basics**: Understand hosting Python apps on cloud platforms.
  - Heroku Docs: https://devcenter.heroku.com/ (Free tier).
  - Vercel Docs: https://vercel.com/docs.
  - Render Docs: https://render.com/docs.

Tips: Practice on LeetCode or Codecademy for Python. Watch tutorials on YouTube (e.g., Corey Schafer for Python, OpenCV official channel). Install libraries: `pip install opencv-python python-Levenshtein tkinter`.

3 Step 2: Gathering Data Sources and Datasets

The project requires minimal datasets due to its rule-based nature, but word lists and video samples are crucial for recommendations and visual input testing. Allocate 3–5 days.

- **Morse Code Mappings**:
  - *Source*: ITU Morse Code Chart.
  - *Download*: https://www.itu.int/dms_pubrec/itu-r/rec/m/r-rec-m.1677-1-200910-i!pdf-e.pdf.
  - *Alternative*: Wikipedia Morse Code page (https://en.wikipedia.org/wiki/Morse_code).
  - *Usage*: Hardcode mappings in Python (e.g., `{'A': '.-', 'B': '-...'}`).
- **English Word List for Recommendations**:
  - *Purpose*: Suggest words for invalid Morse inputs.
  - *Sources*:
    * DWYL English Words: https://github.com/dwyl/english-words (466k+ words, `words.txt`).
    * Kaggle English Word Frequency: https://www.kaggle.com/datasets/rtatman/english-word-frequency (333k words, CSV).
    * Google 10k English: https://github.com/first20hours/google-10000-english (10k common words, TXT).
    * SCOWL Word Lists: http://wordlist.aspell.net/ (Customizable lists).

        * Hugging Face English Words: https://huggingface.co/datasets/Maximax67/English-Valid-Words (173k words, CSV).
    - *Usage*: Load list into Python, precompute Morse codes, use Levenshtein for suggestions.
- **Videos/Datasets for Visual Input Testing**:
    - *Purpose*: Test light flash detection (dots/dashes).
    - *Sources*:
        * GitHub MorseDecoder: https://github.com/new-silvermoon/MorseDecoder (Sample code/videos).
        * YouTube: Search "Morse Code with Flashlight" (e.g., https://www.youtube.com/watch?v=i3HOGdQkTvM, download via yt-dlp).
        * CodeProject: https://www.codeproject.com/Articles/46174/Computer-Vision-De (Sample videos).
        * IRJMETS Eye Blink Morse: https://www.irjmets.com/uploadedfiles/paper//issue_4_april_2025/73109/final/fin_irjmets1745131454.pdf (Adapt for lights).
        * Instructables Arduino Flasher: https://www.instructables.com/Arduino-Morse-Co (Create custom videos).
    - *Usage*: Feed videos to OpenCV; manually label flashes for validation.

Tips: Download datasets from GitHub/Kaggle. Record custom videos with a flashlight for testing.

4 Step 3: Design and Planning

Plan the project structure and architecture:

- **Project Structure**: Create folders: `src/` (code), $\texttt{data/word}_{l}ists(wordlists), \texttt{data/videos}(testv$

- - Morse dictionary from ITU.

- Functions: $\texttt{english}_{t}o_{m}orse(), \texttt{morse}_{t}o_{e}nglish(), \texttt{get}_{w}ord_{s}uggestions()(Levenshtein), \texttt{detect}_{m}orse_{f}ro$

Tools: Use Git for version control (https://github.com).

5 Step 4: Implementation

Implement in phases over 1–2 weeks:

1. **Basic Translation**:
    - Hardcode Morse dictionary from ITU.
- Implement $\texttt{english}_{t}o_{m}orse()and\texttt{morse}_{t}o_{e}nglish().Addvalidation(e.g., regex : \texttt{r'\^{}[./ -]+\$'})$.

2. **Word Recommendations**:
   - Load word list (e.g., DWYL `words.txt`).
   - Use `Levenshtein.distance()` to suggest 4 closest words.
   - Allow user selection of suggested word.
3. **Visual Input**:
   - Use OpenCV (`cv2.VideoCapture(0)`) for webcam/video.
   - Detect flashes: Analyze brightness (`np.mean(cv2.cvtColor(frame, cv2.COLOR`$_B GR2GRA$
     $0.2s = dot, elsedash.$
   - Integrate with translation functions.
4. **GUI (Optional)**:
   - Use Tkinter for buttons (e.g., "Translate", "Start Camera").
   - Display output in text boxes.
5. **Testing**:
   - Unit tests: `pytest` (e.g., `assert english`$_t o_m orse$("SOS") == "... − − − ...").$Edgecases : Invalidinputs, emptystrings, noisyvideos.$
   - Use sample videos from Step 2.

6 Step 5: Deployment

Deploy the application (1–3 days):

- **Platforms**:
  - Heroku: https://devcenter.heroku.com/articles/getting-started-with-python (Free tier).
  - Vercel: https://vercel.com/guides/deploying-python-with-vercel.
  - Render: https://render.com/docs/deploy-python.
  - Alternatives: PythonAnywhere (https://www.pythonanywhere.com), Railway (https://railway.app).
- **Steps**:
  1. Prepare: Create `requirements.txt` (`pip freeze > requirements.txt`), `Procfile` (e.g., `web: python app.py`).
  2. Git Push: Commit to GitHub, link to platform.
  3. Deploy: Use CLI (e.g., `heroku create`, `git push heroku main`).
  4. Test: Access via URL; ensure webcam works (may need HTTPS).

7 Conclusion

This approach outlines a 3–5 week plan to develop a Morse code translation system with English-Morse translation, word recommendations, and visual input processing. By leveraging official resources, open-source datasets, and modular implementation, the project can be efficiently built and deployed for practical use in emergency communication, education, and more.